The most common parameter that a user might tamper with, is the id parameter, as in `http://www.domain.com/project/1`, whereas 1 is the id. It will be available in params in the controller. There, you will most likely do something like this:

```
@project = Project.find(params[:id])
```

This is alright for some web applications, but certainly not if the user is not authorized to view all projects. If the user changes the id to 42, and they are not allowed to see that information, they will have access to it anyway. Instead, *query the user's access rights, too*:

```
@project = @current_user.projects.find(params[:id])
```

Depending on your web application, there will be many more parameters the user can tamper with. As a rule of thumb, *no user input data is secure, until proven otherwise, and every parameter from the user is potentially manipulated*.

Don't be fooled by security by obfuscation and JavaScript security. The Web Developer Toolbar for Mozilla Firefox lets you review and change every form's hidden fields. *JavaScript can be used to validate user input data, but certainly not to prevent attackers from sending malicious requests with unexpected values*. The Live Http Headers plugin for Mozilla Firefox logs every request and may repeat and change them. That is an easy way to bypass any JavaScript validations. And there are even client-side proxies that allow you to intercept any request and response from and to the Internet.

7 Injection

Injection is a class of attacks that introduce malicious code or parameters into a web application in order to run it within its security context. Prominent examples of injection are cross-site scripting (XSS) and SQL injection.

Injection is very tricky, because the same code or parameter can be malicious in one context, but totally harmless in another. A context can be a scripting, query or programming language, the shell or a Ruby/Rails method. The following sections will cover all important contexts where injection attacks may happen. The first section, however, covers an architectural decision in connection with Injection.

7.1 Whitelists versus Blacklists

When sanitizing, protecting or verifying something, prefer whitelists over blacklists.

A blacklist can be a list of bad e-mail addresses, non-public actions or bad HTML tags. This is opposed to a whitelist which lists the good e-mail addresses, public actions, good HTML tags and so on. Although sometimes it is not possible to create a whitelist (in a SPAM filter, for example), *prefer to use whitelist approaches*:

- Use before_action only: [...] instead of except: [...]. This way you don't forget to turn it off for newly added actions.
- Allow instead of removing <script> against Cross-Site Scripting (XSS). See below for details.
- Don't try to correct user input by blacklists:
  - This will make the attack work: "<sc<script>ript>".gsub("<script>", "")
  - But reject malformed input

Whitelists are also a good approach against the human factor of forgetting something in the blacklist.

7.2 SQL Injection

Thanks to clever methods, this is hardly a problem in most Rails applications. However, this is a very devastating and common attack in web applications, so it is important to understand the problem.